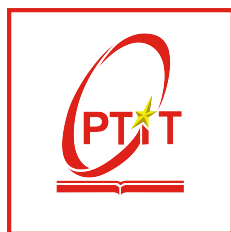


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO THỰC TẬP CƠ SỞ
KHÓA HỌC: THỰC TẬP CƠ SỞ

ĐỀ TÀI: Xây dựng phần mềm Point-of-Sale (POS)
hỗ trợ quản lý bán hàng và đối soát lương nhân sự
cho mô hình dịch vụ

Sinh viên thực hiện:
B23DCCE060 Nguyễn Minh Tuấn Kiệt

Giảng viên hướng dẫn:
Thầy: Kim Ngọc Bách

Class: D23CQCE06 _B

Hà Nội, 06-2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm: (Bằng chữ:)

Hà Nội, ngày tháng năm 2026

Giảng viên hướng dẫn

LỜI CẢM ƠN

Dự án này là kết quả của hơn một học kỳ em tự tìm hiểu và xây dựng, nhưng không phải một mình. Những kiến thức về lập trình web, cơ sở dữ liệu và thiết kế hệ thống mà các thầy cô trong khoa truyền dạy đã được em dùng trực tiếp – từng câu SQL, từng khái niệm về phân quyền, từng nguyên tắc tổ chức code đều xuất hiện đâu đó trong bài này.

Em đặc biệt cảm ơn thầy **Kim Ngọc Bách** đã chỉ ra những chỗ em nghĩ đúng nhưng chưa đúng, giúp em điều chỉnh hướng đi thay vì cứ đi sai mãi.

Dự án chắc chắn còn nhiều chỗ có thể làm tốt hơn. Em mong nhận được góp ý để lần sau làm được tốt hơn lần này.

Em xin chân thành cảm ơn!

Mục lục

Lời cảm ơn	2
1 MỞ ĐẦU	5
1.1 Lý do chọn đề tài	5
1.2 Tính cấp thiết của đề tài	5
1.3 Mục tiêu thực hiện	6
1.4 Phạm vi và giới hạn đề tài	6
2 TỔNG QUAN LÝ THUYẾT VÀ CÁC TÍNH NĂNG CỦA PHẦN MỀM	8
2.1 Khái niệm hệ thống POS	8
2.2 Các công nghệ nền tảng được sử dụng	9
2.3 Tổng quan các tính năng	9
3 PHÂN TÍCH HỆ THỐNG	11
3.1 Các chức năng của hệ thống	11
3.2 Đối tượng người dùng	11
3.3 Yêu cầu chức năng	12
3.4 Yêu cầu phi chức năng	13
4 THIẾT KẾ HỆ THỐNG	15
4.1 Kiến trúc tổng thể	15
4.2 Thiết kế Back-end	15
4.3 Thiết kế Front-end	24
4.4 Thiết kế cơ sở dữ liệu	26
4.5 Thiết kế giao diện hệ thống	30
5 HIỆU QUẢ SỬ DỤNG VÀ HƯỚNG PHÁT TRIỂN	35
5.1 Hiệu quả sử dụng	35

5.2	Hướng phát triển trong tương lai	35
6	KẾT LUẬN	37
6.1	Tóm tắt quá trình hoàn thành	37
6.2	Kết quả đạt được	37
6.3	Đánh giá kết quả	37
	Tài liệu tham khảo	39

Chương 1

MỞ ĐẦU

1.1 Lý do chọn đề tài

Thử hình dung một buổi trưa bận ở một quán ăn nhỏ: nhân viên phục vụ cầm tờ giấy ghi tay, chạy xuống bếp hét order, rồi cuối bữa ngồi cộng tay từng món cho khách. Chậm, dễ nhầm, và khi chủ quán muốn biết hôm nay thu được bao nhiêu, món nào bán được nhiều – gần như không trả lời được nếu chỉ dựa vào đồng hóa đơn giấy.

Phần mềm POS (Point of Sale – phần mềm thu ngân tại điểm bán) giải quyết được những vấn đề đó. Tuy nhiên, các phần mềm thương mại trên thị trường thường có phí thuê bao hàng tháng khá cao và đòi phải mua thêm thiết bị riêng – không phù hợp với quán ăn vừa mới mở hoặc kinh doanh nhỏ.

Vì vậy, em tự xây dựng một phần mềm POS – đặt tên là **Foody** – chạy trên trình duyệt web thông thường, không cần phần cứng chuyên biệt. Đây cũng là cơ hội thực hành đúng nghĩa những gì đã học: giao diện bằng React, phía máy chủ bằng Python/Flask, lưu trữ dữ liệu bằng MySQL.

1.2 Tính cấp thiết của đề tài

Vấn đề của quán ăn nhỏ không chỉ là chậm. Ghi tay dễ nhầm bàn, nhầm món – khách nhận sai đồ ăn, thu sai tiền. Giờ đông, phục vụ chạy đi chạy lại, bếp không biết ưu tiên nấu gì trước. Và cuối ca, chủ quán muốn biết hôm nay thu bao nhiêu, nhân viên nào phục vụ tốt – gần như không trả lời được nếu không có dữ liệu.

Một phần mềm POS giải quyết cùng lúc những vấn đề này: order được ghi vào hệ thống ngay lập tức, bếp thấy trực tiếp trên màn hình, thanh toán tự tính tiền, và mọi giao dịch đều lưu lại để báo cáo sau. Phục vụ – bếp – thu ngân cùng làm việc trên một nguồn dữ liệu, không cần gọi nhau hay chạy đi chạy lại.

1.3 Mục tiêu thực hiện

Mục tiêu chính: Xây dựng một phần mềm POS hoàn chỉnh cho quán ăn nhỏ, bao trọn luồng làm việc thực tế: nhân viên gọi món → bếp nhận và chế biến → thu ngân thanh toán → quản lý xem báo cáo.

Chi tiết hơn:

1. Màn hình bán hàng: thấy được toàn bộ sơ đồ bàn trong quán, gọi món nhanh, tiền tạm tính tự cập nhật ngay trên ô bàn.
2. Các tình huống thực tế: khách đổi bàn thì chuyển hóa đơn, hai nhóm ngồi chung thì gộp bàn, muốn tách tiền riêng thì tách hóa đơn, áp chiết khấu và VAT khi thanh toán.
3. Màn hình bếp: hiển thị hàng đợi món theo thứ tự gọi (ai gọi trước nấu trước – FIFO), nhân viên bếp tự cập nhật trạng thái từng món.
4. Đăng nhập và phân quyền: 4 vai trò khác nhau, mỗi vai trò chỉ làm được những việc của mình.
5. Quản trị: thêm/sửa/xóa thực đơn và danh mục, quản lý nhân viên và tài khoản, ghi sổ thu/chi.
6. Báo cáo: doanh thu theo ngày/tháng, giờ cao điểm, top món bán chạy, hiệu suất từng nhân viên.
7. Giao diện hoàn toàn tiếng Việt, in được hóa đơn ra máy in.

1.4 Phạm vi và giới hạn đề tài

Phần mềm hướng tới: quán ăn, nhà hàng, quán cà phê quy mô nhỏ đến vừa. Toàn bộ chạy trên trình duyệt web, kết nối trong cùng mạng Wi-Fi của quán – không cần Internet ra ngoài.

Những gì chưa làm được:

- **Quản lý kho và nguyên liệu** (tồn kho, cảnh báo hết hàng) – đã cân nhắc nhưng bỏ ra ngoài phạm vi để giữ phần mềm gọn, tập trung vào bán hàng.
- **Thanh toán QR thực tế** – hiện dùng ảnh QR tĩnh để minh họa giao diện, chưa kết nối được cổng thanh toán của ngân hàng.

- **Kiểm thử tự động** – việc kiểm tra tính năng hiện thực hiện thủ công theo quy trình, chưa có bộ test code tự chạy.

Chương 2

TỔNG QUAN LÝ THUYẾT VÀ CÁC TÍNH NĂNG CỦA PHẦN MỀM

2.1 Khái niệm hệ thống POS

POS (Point of Sale) hiểu đơn giản là phần mềm thu ngân tại điểm bán hàng. Nhưng với nhà hàng, nó làm được nhiều hơn việc tính tiền: nhận order từ phục vụ và chuyển ngay xuống bếp, theo dõi từng bàn đang có khách hay còn trống, lưu lại mọi giao dịch để cuối ngày tổng hợp báo cáo.

Nói cách khác, POS thay thế cho tờ giấy ghi tay, chiếc máy tính cầm tay và sổ doanh thu thủ công – nối thông tin giữa phục vụ, bếp, thu ngân và chủ quán trong cùng một chỗ.

2.2 Các công nghệ nền tảng được sử dụng

Bảng 2.1: Các công nghệ nền tảng

Thành phần	Công nghệ	Vai trò
Giao diện (Front-end)	React 19 + Vite	Xây dựng giao diện SPA, render nhanh, hot-reload khi phát triển
Định tuyến	React Router	Điều hướng giữa trang đăng nhập và ứng dụng chính
Gọi API	DOM Axios	Giao tiếp HTTP với máy chủ, tự đính kèm token
Biểu đồ	Recharts	Vẽ biểu đồ doanh thu trong báo cáo
In ấn	react-to-print	In hóa đơn từ giao diện trình duyệt
Máy chủ (Back-end)	Python + Flask	Cung cấp REST API theo mô hình blueprint
Xác thực	JWT (PyJWT) + Werkzeug	Sinh token đăng nhập và băm mật khẩu an toàn
Cơ sở dữ liệu	MySQL	Lưu trữ dữ liệu quan hệ; dùng thêm function/procedure/trigger/view

2.3 Tổng quan các tính năng

Thay vì liệt kê tính năng khô khan, hãy xem phần mềm hoạt động thế nào trong một ca làm việc thực tế:

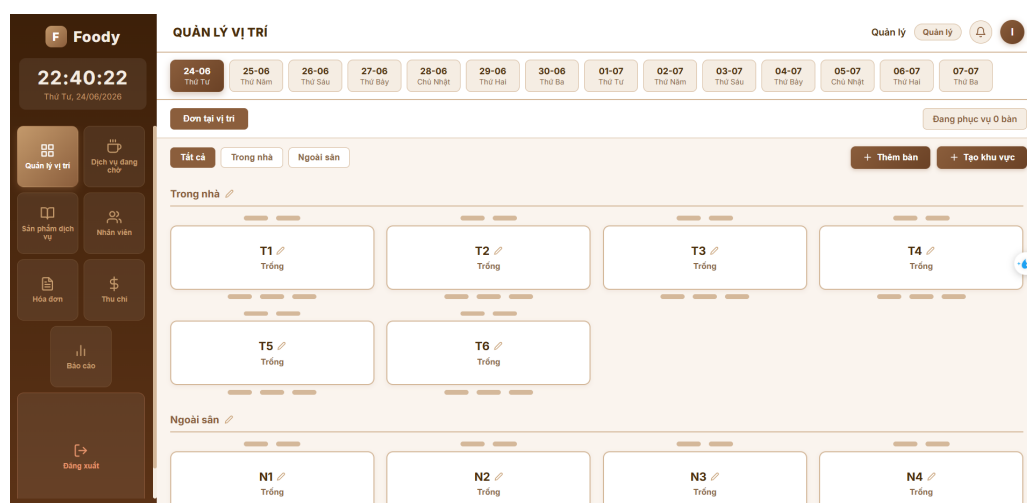
Luồng chính – từ lúc khách vào đến lúc ra:

1. **Phục vụ** đăng nhập, mở màn hình sơ đồ bàn. Bàn trống hiện màu khác với bàn đang có khách, và mỗi bàn đang phục vụ hiển thị luôn số tiền tạm tính. Khi có khách ngồi, phục vụ chọn bàn, tạo hóa đơn và gọi món từ thực đơn theo danh mục.
2. **Bếp** nhìn thấy món vừa được gọi ngay trên màn hình bếp – theo thứ tự thời gian gọi (bàn nào gọi trước hiện trước). Bếp bấm chuyển trạng thái: *Chờ chế biến* → *Đang chế biến* → *Xong*.
3. **Thu ngân** nhận bàn muốn thanh toán: chọn hình thức (tiền mặt hoặc chuyển khoản), nhập chiết khấu nếu có, bấm xác nhận. Bàn tự động trở về trống.
4. **Quản lý** vào xem báo cáo: hôm nay thu bao nhiêu, giờ nào đông nhất, món nào bán được nhiều, từng nhân viên phục vụ bao nhiêu bàn.

Các tình huống thực tế hay gặp:

- Khách muốn đổi bàn → phục vụ chuyển hóa đơn sang bàn mới, toàn bộ món đã gọi đi theo, không mất gì.
- Hai nhóm khách nhập lại một bàn → gộp hóa đơn, tiền tự cộng.
- Một bàn muốn tách tiền riêng → tách món sang hóa đơn mới, giá giữ nguyên như lúc gọi.
- Ghi thêm khoản thu/chi ngoài hóa đơn (nhập hàng, điện nước...) → vào sổ thu/chi, tự tổng kết cuối ngày.

Về bảo mật: Mỗi người đăng nhập tài khoản riêng; phiên đăng nhập hết hạn sau 12 giờ và hệ thống tự đăng xuất. Vai trò quyết định người đó thấy và làm được gì – nhân viên bếp chỉ thấy màn hình bếp, phục vụ không bấm được nút thanh toán.



Hình 2.1: Giao diện tổng thể ứng dụng Foody

Chương 3

PHÂN TÍCH HỆ THỐNG

3.1 Các chức năng của hệ thống

Phần mềm được tổ chức thành các nhóm chức năng riêng biệt, mỗi nhóm tương ứng một file Python ở phía máy chủ:

Bảng 3.1: Các phân hệ chức năng của hệ thống

Phân hệ	Chức năng chính
Xác thực & Phân quyền	Đăng nhập, lấy thông tin người dùng hiện tại, kiểm soát quyền truy cập
POS (bán hàng)	Quản lý khu vực, bàn, hóa đơn, gọi món, chuyển/gộp bàn, tách hóa đơn, thanh toán
Bếp	Hàng đợi món FIFO, cập nhật trạng thái chế biến
Quản trị thực đơn	CRUD món ăn và danh mục
Quản trị nhân sự	CRUD nhân viên + tài khoản, đặt lại mật khẩu
Thu/Chi	Tạo, xem, xóa phiếu thu/chi
Báo cáo	Tổng quan, doanh thu, theo tháng, hiệu suất nhân viên

3.2 Đối tượng người dùng

Phần mềm có 4 loại tài khoản, mỗi loại thấy và làm được những gì phù hợp với công việc của mình. Tài khoản **Quản lý** có thể làm tất cả mọi thứ mà các vai trò khác làm được.

Bảng 3.2: Các đối tượng người dùng và quyền hạn

Vai trò	Mô tả	Quyền hạn chính
Quản lý	Chủ quán / người quản lý	Toàn quyền: bán hàng, quản trị thực đơn, nhân sự, khu vực/bàn, xem báo cáo, xóa phiếu thu chi
Thu ngân	Nhân viên thu ngân	Bán hàng, thanh toán, thao tác bàn/khu vực, thu chi, xem hàng đợi bếp
Phục vụ	Nhân viên phục vụ	Gọi món, chuyển/gộp/tách hóa đơn, thu chi, xem hàng đợi bếp (không được thanh toán)
Bếp	Nhân viên bếp	Xem hàng đợi món và cập nhật trạng thái chế biến; mặc định vào thẳng màn hình Bếp

Cách phân nhóm quyền trong code: Thu ngân và Phục vụ được gộp thành SALES_ROLES – cả hai đều làm được các việc bán hàng thông thường. Riêng thanh toán chỉ mở cho Thu ngân (CASHIER_ROLES). Mọi thao tác thêm/sửa/xóa dữ liệu (khu vực, bàn, thực đơn, nhân viên) chỉ Quản lý mới được phép.

3.3 Yêu cầu chức năng

FR1 – Đăng nhập: Người dùng nhập tên và mật khẩu. Nếu đúng, hệ thống trả về một token JWT – hiểu như một “vé” điện tử ghi rõ vai trò của người đó, hết hạn sau 12 giờ. Mọi thao tác sau đó đều phải kèm theo vé này; nếu hết hạn hoặc không hợp lệ, hệ thống từ chối và đăng xuất.

FR2 – Sơ đồ bàn: Hiển thị toàn bộ khu vực và bàn. Bàn đang có khách hiện khác màu và cho thấy số tiền tạm tính. Quản lý có thể thêm/sửa/xóa khu vực và bàn – trừ bàn đang có hóa đơn mở (tránh xóa nhầm khi đang phục vụ).

FR3 – Gọi món: Tạo hóa đơn cho bàn rồi gọi món từ thực đơn. Nếu gọi thêm một món đã có, số lượng tự tăng lên thay vì tạo dòng mới. Giảm số lượng về 0 thì món tự xóa khỏi hóa đơn. Tiền tạm tính theo bàn cập nhật liên tục.

FR4 – Chuyển/Gộp/Tách bàn:

- *Chuyển bàn:* toàn bộ hóa đơn chuyển sang bàn mới – dùng khi khách muốn đổi chỗ.
- *Gộp bàn:* chuyển hóa đơn vào một bàn đang có khách – hai nhóm ngồi chung một bàn.
- *Tách hóa đơn:* chọn một số món (kèm số lượng) để tách sang hóa đơn mới trên cùng bàn – dùng khi cùng bàn muốn thanh toán riêng.

FR5 – Thanh toán: Thu ngân chọn phương thức (tiền mặt hoặc chuyển khoản), nhập chiết khấu và VAT nếu có. Tiền thực thu tính theo công thức:

$$\text{Tiền thực thu} = \text{Tạm tính} \times (1 - \text{Chiết khấu}\%) \times (1 + \text{VAT}\%)$$

Sau khi xác nhận, hóa đơn chuyển sang “Đã thanh toán” và bàn tự trống lại – không cần thao tác thêm.

FR6 – Màn hình bếp: Hiện toàn bộ món của các bàn đang phục vụ, sắp xếp theo thứ tự thời gian gọi (ai gọi trước hiện trước). Nhân viên bếp bấm cập nhật trạng thái từng món qua 3 bước: *Chờ chế biến* → *Đang chế biến* → *Xong*.

FR7 – Quản trị: Quản lý thêm/sửa/xóa thực đơn, danh mục món, khu vực và bàn. Khi tạo nhân viên mới, tài khoản đăng nhập được tạo cùng lúc. Đặt lại mật khẩu được tách riêng để tránh vô tình ghi đè.

FR8 – Sổ thu/chi: Ghi các khoản thu/chi ngoài hóa đơn (nhập hàng, điện nước, tiền lẻ thu thêm...). Phục vụ và Thu ngân đều tạo được phiếu; xóa phiếu chỉ Quản lý được phép. Tổng thu và tổng chi tự cộng dồn.

FR9 – Báo cáo: Quản lý lọc theo khoảng ngày để xem: tổng doanh thu và số hóa đơn, phân theo phương thức thanh toán, biểu đồ theo giờ trong ngày (để biết giờ nào đông khách nhất), top 3 món bán chạy, và hiệu suất từng nhân viên.

3.4 Yêu cầu phi chức năng

- **Bảo mật:** Mật khẩu không bao giờ lưu dưới dạng văn bản thường – được băm bằng thuật toán Werkzeug trước khi vào cơ sở dữ liệu. Phân quyền được kiểm tra ở cả hai phía: phía máy chủ (decorator `require_role`) từ chối nếu vai trò không đủ, và phía giao diện (`hasRole`) ẩn nút/tab không được phép dùng.
- **Không rò rỉ kết nối:** Mỗi lần có yêu cầu từ người dùng, hệ thống mở một kết nối MySQL mới, dùng xong thì đóng lại ngay. Cách này tránh tình trạng giữ quá nhiều kết nối mở cùng lúc khi quán đông.
- **Giao diện không bị dừng khi mạng chập chờn:** Nếu một thao tác ghi thất bại do lỗi mạng tạm thời, giao diện vẫn phản hồi ngay cho người dùng thay vì đứng chờ – tuy nhiên dữ liệu sẽ không được lưu lại nếu không kết nối lại được.
- **Dữ liệu nhất quán dù nhiều người cùng thao tác:** Việc đóng hóa đơn và trả bàn về trống được xử lý bởi trigger trong cơ sở dữ liệu – điều này đảm bảo dù phục vụ và thu ngân cùng thao tác lúc đó, bàn vẫn chỉ trống đúng một lần.

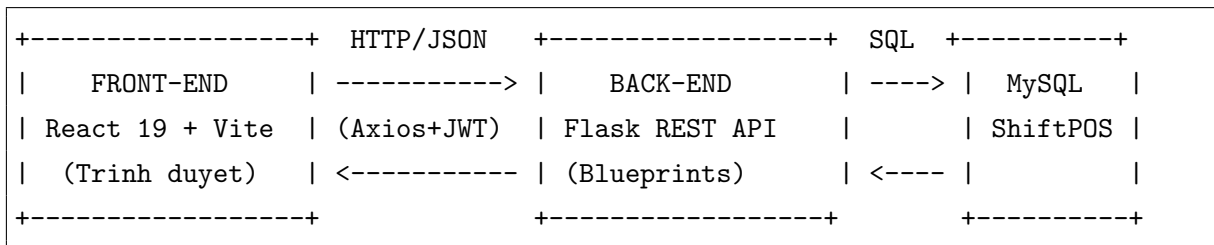
- **Dễ bảo trì:** Mỗi nhóm chức năng một file riêng, cấu hình kết nối (mật khẩu DB, cổng, khóa JWT) đặt qua biến môi trường thay vì ghi cứng trong code.

Chương 4

THIẾT KẾ HỆ THỐNG

4.1 Kiến trúc tổng thể

Phần mềm chia thành 3 tầng rõ ràng – mỗi tầng làm một việc, không xâm phạm vào tầng kia:



Khi người dùng bấm một nút trên giao diện, luồng diễn ra như sau: giao diện gửi yêu cầu HTTP đến máy chủ Flask, kèm theo token JWT trong header. Flask kiểm tra token và vai trò, nếu hợp lệ thì chạy hàm xử lý tương ứng. Hàm đó dùng `db.py` để truy vấn MySQL và trả kết quả về dạng JSON. Giao diện nhận JSON và cập nhật màn hình. Nếu token hết hạn hoặc bị từ chối (lỗi 401), giao diện tự xóa thông tin đăng nhập và chuyển về trang đăng nhập – người dùng không thấy màn hình trắng hay lỗi khó hiểu.

4.2 Thiết kế Back-end

Phía máy chủ dùng **Flask** (Python). Thay vì nhét mọi thứ vào một file, code được tách ra thành từng file nhỏ theo chức năng – mỗi file gọi là một *blueprint*. File `app.py` chỉ có nhiệm vụ gom các blueprint lại và khởi động server:

Bảng 4.1: Danh sách các blueprint back-end

File	Tiền tố URL	Nghiệp vụ
auth.py	/api/auth	Đăng nhập, thông tin người dùng, decorator phân quyền
pos.py	/api	Khu vực, bàn, thực đơn, hóa đơn, gọi món, chuyển/gộp/tách, thanh toán
kitchen.py	/api/kitchen	Hàng đợi bếp, cập nhật trạng thái món
admin.py	/api/admin	CRUD thực đơn & danh mục
staff.py	/api/staff	CRUD nhân viên & tài khoản
thuchi.py	/api/thuchi	Sổ thu/chi
reports.py	/api/reports	Báo cáo, thống kê

Ngoài ra còn có:

- `config.py`: đọc cấu hình (mật khẩu DB, cổng, khóa JWT...) từ biến môi trường – để không ghi cứng thông tin nhạy cảm trong code.
- `db.py`: lớp kết nối MySQL. Mỗi request vào, file này mở một kết nối mới; xong thì đóng lại. Các hàm `query` (lấy nhiều dòng), `query_one` (lấy 1 dòng), `execute` (ghi và commit) được dùng ở mọi nơi trong code thay vì viết SQL lặp đi lặp lại.
- `init_db.py`: chạy một lần duy nhất để tạo toàn bộ bảng và dữ liệu mẫu – thay thế cho việc copy/paste SQL vào MySQL Workbench.

Listing 4.1: File `db.py` – Lớp truy cập CSDL: quản lý kết nối và các helper query/execute

```

1 # db.py -- Quản lý kết nối MySQL theo từng request (lưu trong flask.g)
2 import mysql.connector
3 from flask import g
4 from config import Config
5
6 def get_conn():
7     """Lay ket noi MySQL cho request hien tai; tao moi neu chua co."""
8     if "db_conn" not in g:
9         g.db_conn = mysql.connector.connect(
10             host=Config.DB_HOST, port=Config.DB_PORT,
11             user=Config.DB_USER, password=Config.DB_PASSWORD,
12             database=Config.DB_NAME, charset="utf8mb4", autocommit=False,
13         )
14     return g.db_conn
15
16 def close_conn(_exc=None):
17     conn = g.pop("db_conn", None)
18     if conn is not None:
19         try:
20             conn.close()
21         except Exception:

```

```

22         pass
23
24 def query(sql, params=None):
25     """SELECT nhieu dong, tra ve list[dict]."""
26     cur = get_conn().cursor(dictionary=True)
27     cur.execute(sql, params or ())
28     rows = cur.fetchall(); cur.close()
29     return rows
30
31 def query_one(sql, params=None):
32     """SELECT 1 dong, tra ve dict hoac None."""
33     cur = get_conn().cursor(dictionary=True)
34     cur.execute(sql, params or ())
35     row = cur.fetchone(); cur.close()
36     return row
37
38 def execute(sql, params=None):
39     """INSERT / UPDATE / DELETE: commit va tra ve lastrowid."""
40     conn = get_conn()
41     cur = conn.cursor()
42     cur.execute(sql, params or ())
43     last_id = cur.lastrowid
44     cur.close(); conn.commit()
45     return last_id
46
47 def callproc(proc_name, args=()):
48     """Goi stored procedure, commit, tra ve ket qua."""
49     conn = get_conn()
50     cur = conn.cursor(dictionary=True)
51     cur.callproc(proc_name, args)
52     rows = cur.fetchall(); cur.close(); conn.commit()
53     return rows
54
55 def init_app(app):
56     app.teardown_appcontext(close_conn)

```

Listing 4.2: File app.py – App-factory: đăng ký blueprint và khởi tạo Flask

```

1 from flask import Flask, jsonify
2 from flask_cors import CORS
3 import db
4 from config import Config
5 from auth import auth_bp
6 from pos import pos_bp
7 from kitchen import kitchen_bp
8 from admin import admin_bp
9 from reports import reports_bp
10 from staff import staff_bp
11 from thuchi import thuchi_bp
12
13 def create_app():
14     app = Flask(__name__)
15     app.config.from_object(Config)
16
17     CORS(app)          # cho phép frontend (Vite dev server) gọi API
18     db.init_app(app)    # dong ket noi MySQL khi ket thuc request
19
20     app.register_blueprint(auth_bp)
21     app.register_blueprint(pos_bp)

```

```

22     app.register_blueprint(kitchen_bp)
23     app.register_blueprint(admin_bp)
24     app.register_blueprint(reports_bp)
25     app.register_blueprint(staff_bp)
26     app.register_blueprint(thuchi_bp)
27
28     @app.route("/api/health")
29     def health():
30         return jsonify({"status": "ok"})
31
32     @app.errorhandler(400)
33     def bad_request(_e):
34         return jsonify({"error": "Yeu cau khong hop le"}), 400
35
36     @app.errorhandler(404)
37     def not_found(_e):
38         return jsonify({"error": "Khong tim thay"}), 404
39
40     @app.errorhandler(500)
41     def server_error(_e):
42         return jsonify({"error": "Loi may chu noi bo"}), 500
43
44     return app
45
46 app = create_app()
47
48 if __name__ == "__main__":
49     app.run(host=Config.HOST, port=Config.PORT, debug=Config.DEBUG)

```

Cách hoạt động của phân quyền:

Khi đăng nhập thành công, server tạo một token JWT – bên trong token ghi tên đăng nhập, mã nhân viên và vai trò. Token này gửi kèm vào mọi request sau đó như một cái vé để chứng minh danh tính.

Mỗi hàm API được gắn một decorator `@require_role(...)`. Khi có request đến, decorator đọc token, lấy vai trò ra và so sánh. Viết `@require_role()` không có tham số nghĩa là chỉ Quản lý mới vào được. Quản lý thì không cần khai báo riêng – code tự cho qua tất cả.

Listing 4.3: File `auth.py` – Decorator `require_role` (RBAC) và endpoint đăng nhập

```

1  # auth.py -- Xac thuc & Phan quyen theo vai tro (RBAC)
2  from functools import wraps
3  import jwt
4  from flask import Blueprint, request, jsonify, g
5  from werkzeug.security import check_password_hash
6  from config import Config
7  from db import query_one
8
9  auth_bp = Blueprint("auth", __name__, url_prefix="/api/auth")
10
11 def require_auth(fn):
12     """Yeu cau token JWT hop le; gan thong tin user vao g.current_user."""
13     @wraps(fn)
14     def wrapper(*args, **kwargs):
15         token = (request.headers.get("Authorization", "") or "")[7:].strip()

```

```

16         if not token:
17             return jsonify({"error": "Thieu token xac thuc"}), 401
18         try:
19             g.current_user = jwt.decode(
20                 token, Config.JWT_SECRET, algorithms=[Config.JWT_ALGORITHM])
21         except jwt.ExpiredSignatureError:
22             return jsonify({"error": "Phien dang nhap da het han"}), 401
23         except jwt.InvalidTokenError:
24             return jsonify({"error": "Token khong hop le"}), 401
25         return fn(*args, **kwargs)
26     return wrapper
27
28 def require_role(*roles):
29     """Kiem tra vai tro. 'Quan ly' luon duoc phep (toan quyen).
30     Goi khong tham so: require_role() = chi Quan ly."""
31     def decorator(fn):
32         @wraps(fn)
33         @require_auth
34         def wrapper(*args, **kwargs):
35             vai_tro = g.current_user.get("VaiTro")
36             if vai_tro != "Quan ly" and vai_tro not in roles:
37                 return jsonify({"error": "Ban khong co quyen truy cap"}), 403
38             return fn(*args, **kwargs)
39         return wrapper
40     return decorator
41
42 @auth_bp.route("/login", methods=["POST"])
43 def login():
44     data = request.get_json(silent=True) or {}
45     username = (data.get("TenDangNhap") or "").strip()
46     password = data.get("MatKhau") or ""
47     row = query_one(
48         "SELECT TK.TenDangNhap, TK.MatKhau, NV.MaNV, NV.TenNV, NV.VaiTro "
49         "FROM TAIKHOAN TK JOIN NHANVIEN NV ON TK.MaNV = NV.MaNV "
50         "WHERE TK.TenDangNhap = %s", (username,))
51     if not row or not check_password_hash(row["MatKhau"], password):
52         return jsonify({"error": "Sai ten dang nhap hoac mat khau"}), 401
53     token = jwt.encode(
54         {"TenDangNhap": row["TenDangNhap"],
55          "MaNV": row["MaNV"], "VaiTro": row["VaiTro"]},
56         Config.JWT_SECRET, algorithm=Config.JWT_ALGORITHM)
57     return jsonify({"token": token,
58                    "user": {"TenDangNhap": row["TenDangNhap"],
59                             "MaNV": row["MaNV"], "TenNV": row["TenNV"],
60                             "VaiTro": row["VaiTro"]}})

```

Listing 4.4: File pos.py – Route thanh toán POST /api/checkout/{maHd}

```

1  # pos.py -- Dinh nghia nhom quyen
2  SALES_ROLES = ("Thu ngan", "Phuc vu")      # nghiep vu ban hang
3  CASHIER_ROLES = ("Thu ngan",)              # chi thu ngan duoc thanh toan
4
5  # POST /api/checkout/<mahd> -- Thanh toan hoa don
6  @pos_bp.route("/checkout/<int:mahd>", methods=["POST"])
7  @require_role(*CASHIER_ROLES)
8  def checkout(mahd):
9      data = request.get_json(silent=True) or {}
10     pttt = data.get("PhuongThucTT", "Tien mat")
11     chiet_khau = float(data.get("ChietKhau", 0) or 0) # phan tram

```

```

12     thue_vat      = float(data.get("ThueVAT", 0) or 0)      # phan tram
13
14     inv = query_one("SELECT TrangThaiHD FROM HOADON WHERE MaHD=%s", (mahd,))
15     if not inv:
16         return jsonify({"error": "Khong tim thay hoa don"}), 404
17     if inv["TrangThaiHD"] == "Da thanh toan":
18         return jsonify({"error": "Hoa don da duoc thanh toan"}), 409
19
20     # Trigger trg_DonBan_SauThanhToan tu set ThoiGianDong va don ban
21     execute(
22         "UPDATE HOADON SET TrangThaiHD='Da thanh toan', "
23         "PhuongThucTT=%s, ChietKhau=%s, ThueVAT=%s WHERE MaHD=%s",
24         (pttt, chiet_khau, thue_vat, mahd),
25     )
26     return jsonify({"message": "Thanh toan thanh cong"})

```

Listing 4.5: File pos.py – Route gọi món: thêm và giảm số lượng món trong hóa đơn

```

1  # POST /api/order -- Them mon vao hoa don (tang SoLuong neu mon da co)
2  @pos_bp.route("/order", methods=["POST"])
3  @require_role(*SALES_ROLES)
4  def add_order():
5      data = request.get_json(silent=True) or {}
6      mahd, mamon = data.get("MaHD"), data.get("MaMon")
7      if not mahd or not mamon:
8          return jsonify({"error": "Thieu MaHD hoac MaMon"}), 400
9      mon = query_one("SELECT DonGia FROM MONAN WHERE MaMon=%s", (mamon,))
10     if not mon:
11         return jsonify({"error": "Khong tim thay mon"}), 404
12     # ON DUPLICATE KEY: tang SoLuong, reset TrangThaiMon ve "Cho che bien"
13     execute(
14         "INSERT INTO CHITIET_HD "
15         " (MaHD, MaMon, SoLuong, DonGiaLucBan, TrangThaiMon, ThoiGianGoi) "
16         "VALUES (%s, %s, 1, %s, 'Cho che bien', NOW()) "
17         "ON DUPLICATE KEY UPDATE SoLuong = SoLuong + 1, "
18         " TrangThaiMon='Cho che bien', ThoiGianGoi=NOW()",
19         (mahd, mamon, mon["DonGia"]),
20     )
21     return jsonify({"message": "Da them mon"})
22
23 # POST /api/order/decrease -- Giam so luong; xoa dong neu ve 0
24 @pos_bp.route("/order/decrease", methods=["POST"])
25 @require_role(*SALES_ROLES)
26 def decrease_order():
27     data = request.get_json(silent=True) or {}
28     mahd, mamon = data.get("MaHD"), data.get("MaMon")
29     row = query_one(
30         "SELECT SoLuong FROM CHITIET_HD WHERE MaHD=%s AND MaMon=%s",
31         (mahd, mamon)
32     )
33     if not row:
34         return jsonify({"error": "Mon khong co trong hoa don"}), 404
35     if row["SoLuong"] <= 1:
36         execute("DELETE FROM CHITIET_HD WHERE MaHD=%s AND MaMon=%s",
37             (mahd, mamon))
38     else:
39         execute("UPDATE CHITIET_HD SET SoLuong = SoLuong - 1 "
40             "WHERE MaHD=%s AND MaMon=%s", (mahd, mamon))
41     return jsonify({"message": "Da giam so luong"})

```

Listing 4.6: File kitchen.py – Hàng đợi bếp FIFO và cập nhật trạng thái món

```

1  # kitchen.py -- Man hình Bep: hang doi mon theo thu tu thoi gian (FIFO)
2  from flask import Blueprint, request, jsonify
3  from auth import require_role
4  from db import query, query_one, execute
5
6  kitchen_bp = Blueprint("kitchen", __name__, url_prefix="/api/kitchen")
7  KITCHEN_ROLES = ("Bep", "Phuc vu", "Thu ngan")
8  VALID_STATUS = ("Cho che bien", "Dang che bien", "Xong")
9
10 @kitchen_bp.route("/queue", methods=["GET"])
11 @require_role(*KITCHEN_ROLES)
12 def queue():
13     """Tat ca mon cua hoa don CHUA thanh toan, sap xep FIFO (ThoiGianGoi ASC)."""
14     rows = query(
15         "SELECT CT.MaHD, CT.MaMon, M.TenMon, DM.TenDM AS Loai, "
16         "CT.SoLuong, CT.TrangThaiMon, CT.ThoiGianGoi, H.MaBan, B.TenBan "
17         "FROM CHITIET_HD CT "
18         "JOIN HOADON H ON CT.MaHD = H.MaHD "
19         "JOIN MONAN M ON CT.MaMon = M.MaMon "
20         "LEFT JOIN DANHMUC DM ON M.MaDM = DM.MaDM "
21         "JOIN BAN B ON H.MaBan = B.MaBan "
22         "WHERE H.TrangThaiHD = 'Chua thanh toan' "
23         "ORDER BY CT.ThoiGianGoi ASC"
24     )
25     return jsonify(rows)
26
27 @kitchen_bp.route("/update_status", methods=["POST"])
28 @require_role(*KITCHEN_ROLES)
29 def update_status():
30     data = request.get_json(silent=True) or {}
31     mahd = data.get("MaHD")
32     mamon = data.get("MaMon")
33     trang_thai = data.get("TrangThaiMon")
34     if not mahd or not mamon or trang_thai not in VALID_STATUS:
35         return jsonify({"error": "Du lieu khong hop le"}), 400
36     if not query_one(
37         "SELECT 1 FROM CHITIET_HD WHERE MaHD=%s AND MaMon=%s", (mahd, mamon)
38     ):
39         return jsonify({"error": "Khong tim thay mon trong hoa don"}), 404
40     execute(
41         "UPDATE CHITIET_HD SET TrangThaiMon=%s WHERE MaHD=%s AND MaMon=%s",
42         (trang_thai, mahd, mamon),
43     )
44     return jsonify({"message": "Da cap nhat trang thai mon"})

```

Bảng 4.2: Một số API tiêu biểu

Phương thức	Đường dẫn	Quyền	Mô tả
POST	/api/auth/login	công khai	Đăng nhập
GET	/api/areas, /api/tables, /api/menu	sales	Lấy dữ liệu nền
POST	/api/invoice/create/{maBan}	sales	Tạo hóa đơn
POST	/api/order / /api/order/decrease	sales	Thêm/giảm món
POST	/api/tables/move_merge	sales	Chuyển/gộp bàn
POST	/api/invoice/split	sales	Tách hóa đơn
POST	/api/checkout/{maHd}	cashier	Thanh toán
GET/POST	/api/kitchen/queue, /update_status	bếp	Hàng đợi & trạng thái món
GET	/api/reports/overview	Quản lý	Báo cáo tổng quan

Listing 4.7: File thuchi.py – Sổ thu/chi: tạo, liệt kê và xóa phiếu

```

1 # thuchi.py -- So Thu/Chi (ghi nhan cac khoan thu/chi ngoai hoa don)
2 from flask import Blueprint, request, jsonify, g
3 from auth import require_role
4 from db import query, query_one, execute
5
6 thuchi_bp = Blueprint("thuchi", __name__, url_prefix="/api/thuchi")
7 LEDGER_ROLES = ("Thu ngan", "Phuc vu")
8
9 def _current_manv():
10     user = getattr(g, "current_user", None)
11     return user.get("MaNV") if user else None
12
13 # GET /api/thuchi -- Lay danh sach phieu + tong thu/chi
14 @thuchi_bp.route("", methods=["GET"])
15 @require_role(*LEDGER_ROLES)
16 def list_thuchi():
17     rows = query(
18         """SELECT TC.MaPhieu, TC.LoaiPhieu, TC.SoTien, TC.LyDo,
19             TC.ThoiGian, TC.NguoiLap, NV.TenNV
20             FROM THUCHI TC
21             LEFT JOIN NHANVIEN NV ON TC.NguoiLap = NV.MaNV
22             ORDER BY TC.ThoiGian DESC"""
23     )
24     tong_thu = sum((r["SoTien"] or 0) for r in rows if r["LoaiPhieu"] == "Thu")
25     tong_chi = sum((r["SoTien"] or 0) for r in rows if r["LoaiPhieu"] == "Chi")
26     return jsonify({"items": rows, "TongThu": tong_thu, "TongChi": tong_chi})
27
28 # POST /api/thuchi -- Tao phieu (NguoiLap lay tu JWT token)
29 @thuchi_bp.route("", methods=["POST"])
30 @require_role(*LEDGER_ROLES)
31 def create_thuchi():

```

```

32 d = request.get_json(silent=True) or {}
33 loai = (d.get("LoaiPhieu") or "").strip()
34 lydo = (d.get("LyDo") or "").strip()
35 try:
36     sotien = int(d.get("SoTien") or 0)
37 except (ValueError, TypeError):
38     return jsonify({"error": "So tien khong hop le"}), 400
39 if loai not in ("Thu", "Chi"):
40     return jsonify({"error": "LoaiPhieu phai la 'Thu' hoac 'Chi'"}), 400
41 if sotien <= 0:
42     return jsonify({"error": "So tien phai lon hon 0"}), 400
43 maphieu = execute(
44     "INSERT INTO THUCHI (LoaiPhieu, SoTien, LyDo, NgoaiLap) "
45     "VALUES (%s,%s,%s,%s)",
46     (loai, sotien, lydo, _current_manv()),
47 )
48 return jsonify({"message": "Tao phieu thanh cong", "MaPhieu": maphieu}), 201
49
50 # DELETE /api/thuchi/<maphieu> -- Chi Quan ly duoc xoa phieu
51 @thuchi_bp.route("/<int:maphieu>", methods=["DELETE"])
52 @require_role()
53 def delete_thuchi(maphieu):
54     if not query_one("SELECT 1 FROM THUCHI WHERE MaPhieu=%s", (maphieu,)):
55         return jsonify({"error": "Khong tim thay phieu"}), 404
56     execute("DELETE FROM THUCHI WHERE MaPhieu=%s", (maphieu,))
57     return jsonify({"message": "Da xoa phieu"})

```

Listing 4.8: File `staff.py` – Quản lý nhân viên và tài khoản (chỉ Quản lý)

```

1 # staff.py -- CRUD Nhan vien + Tai khoan, chi Quan ly truy cap
2 from flask import Blueprint, request, jsonify
3 from werkzeug.security import generate_password_hash
4 from auth import require_role
5 from db import query, query_one, execute
6
7 staff_bp = Blueprint("staff", __name__, url_prefix="/api/staff")
8 VALID_ROLES = ("Quan ly", "Thu ngan", "Phuc vu", "Bep")
9
10 # GET /api/staff -- Danh sach nhan vien kem ten dang nhap
11 @staff_bp.route("", methods=["GET"])
12 @require_role("Phuc vu", "Thu ngan", "Bep")
13 def list_staff():
14     return jsonify(query(
15         "SELECT NV.MaNV, NV.TenNV, NV.VaiTro, NV.MucLuong, TK.TenDangNhap "
16         "FROM NHANVIEN NV LEFT JOIN TAIKHOAN TK ON NV.MaNV=TK.MaNV "
17         "ORDER BY NV.MaNV"
18     ))
19
20 # POST /api/staff -- Tao nhan vien + tai khoan cung luc
21 @staff_bp.route("", methods=["POST"])
22 @require_role()
23 def create_staff():
24     d = request.get_json(silent=True) or {}
25     manv = d.get("MaNV", "").strip()
26     tennv = d.get("TenNV", "").strip()
27     vaiTro = d.get("VaiTro", "").strip()
28     tendangnhap = d.get("TenDangNhap", "").strip()
29     matkhau = d.get("MatKhau", "").strip()
30     muluong = int(d.get("MucLuong", 5000000) or 5000000)

```



```

31     if not manv or not tennv or vaitro not in VALID_ROLES:
32         return jsonify({"error": "Thieu thong tin hoac VaiTro khong hop le"}), 400
33     if not tendangnhap or len(matkhau) < 6:
34         return jsonify({"error": "Thieu TenDangNhap hoac mat khau < 6 ky tu"}), 400
35     if query_one("SELECT 1 FROM NHANVIEN WHERE MaNV=%s", (manv,)):
36         return jsonify({"error": "Ma nhan vien da ton tai"}), 409
37     if query_one("SELECT 1 FROM TAIKHOAN WHERE TenDangNhap=%s", (tendangnhap,)):
38         return jsonify({"error": "Ten dang nhap da ton tai"}), 409
39     execute(
40         "INSERT INTO NHANVIEN (MaNV, TenNV, VaiTro, MucLuong) VALUES (%s,%s,%s,%s)",
41         (manv, tennv, vaitro, muluong),
42     )
43     execute(
44         "INSERT INTO TAIKHOAN (TenDangNhap, MatKhau, MaNV) VALUES (%s,%s,%s)",
45         (tendangnhap, generate_password_hash(matkhau), manv),
46     )
47     return jsonify({"message": "Tao nhan vien thanh cong", "MaNV": manv}), 201
48
49 # DELETE /api/staff/<manv> -- Xoa nhan vien (khong cho xoa neu da co hoa don)
50 @staff_bp.route("/<manv>", methods=["DELETE"])
51 @require_role()
52 def delete_staff(manv):
53     if not query_one("SELECT 1 FROM NHANVIEN WHERE MaNV=%s", (manv,)):
54         return jsonify({"error": "Khong tim thay nhan vien"}), 404
55     if query_one("SELECT 1 FROM HOADON WHERE MaNV=%s", (manv,)):
56         return jsonify({"error": "Nhan vien da co hoa don, khong the xoa"}), 409
57     execute("DELETE FROM TAIKHOAN WHERE MaNV=%s", (manv,))
58     execute("DELETE FROM NHANVIEN WHERE MaNV=%s", (manv,))
59     return jsonify({"message": "Da xoa nhan vien"})
60
61 # PUT /api/staff/<manv>/password -- Quan ly dat lai mat khau
62 @staff_bp.route("/<manv>/password", methods=["PUT"])
63 @require_role()
64 def change_password(manv):
65     new_pw = (request.get_json(silent=True) or {}).get("MatKhau", "").strip()
66     if len(new_pw) < 6:
67         return jsonify({"error": "Mat khau phai co it nhat 6 ky tu"}), 400
68     execute(
69         "UPDATE TAIKHOAN SET MatKhau=%s WHERE MaNV=%s",
70         (generate_password_hash(new_pw), manv),
71     )
72     return jsonify({"message": "Da doi mat khau thanh cong"})

```

4.3 Thiết kế Front-end

Giao diện được xây bằng **React 19** – toàn bộ là một trang web duy nhất, khi bấm nút chỉ cập nhật phần cần thiết mà không tải lại trang. Điều này giúp ứng dụng phản hồi nhanh như app trên máy tính bàn.

Các file chính:

- `main.jsx`: điểm khởi động. Ở đây có `ProtectedRoute` – nếu chưa đăng nhập mà cố vào trang chính, sẽ bị chuyển về trang đăng nhập ngay.

- `App.jsx`: màn hình chính sau khi đăng nhập – chứa thanh tab (POS, Bếp, Hóa đơn, Thực đơn, Nhân viên, Báo cáo) và toàn bộ các popup nghiệp vụ (gọi món, thanh toán, chuyển bàn...).
- `AuthContext.jsx`: nơi lưu thông tin người đang đăng nhập và cung cấp hàm `hasRole()` cho toàn ứng dụng.
- `api.js`: cầu nối với máy chủ – tự gắn token vào mọi request và tự xử lý khi token hết hạn.
- `components/`: các màn hình độc lập – đăng nhập, bếp, hóa đơn, thu/chi, quản lý thực đơn, quản lý nhân viên.

Phân quyền trên giao diện: Hàm `hasRole()` kiểm tra vai trò người đang đăng nhập trước khi hiện một tab hay một nút. Ví dụ: tab “Nhân viên” chỉ render khi đang dùng tài khoản Quản lý; tài khoản Bếp đăng nhập vào thì ứng dụng tự nhảy thẳng sang tab Bếp.

Listing 4.9: File `api.js` – Axios instance dùng chung: tự đính JWT và tự đăng xuất khi hết hạn

```

1  import axios from 'axios';
2
3  // Một axios instance dùng chung cho toàn app.
4  // Tu dính kèm JWT token và tu đăng xuất khi token hết hạn (401).
5  const api = axios.create({
6    baseURL: 'http://127.0.0.1:5000',
7  });
8
9  // Request interceptor: dính Bearer token vào mọi request
10 api.interceptors.request.use((config) => {
11   const token = localStorage.getItem('token');
12   if (token) {
13     config.headers.Authorization = `Bearer ${token}`;
14   }
15   return config;
16 });
17
18 // Response interceptor: bắt lỗi 401 -> tu xóa token và chuyển về login
19 api.interceptors.response.use(
20   (res) => res,
21   (err) => {
22     if (err.response && err.response.status === 401) {
23       localStorage.removeItem('token');
24       localStorage.removeItem('user');
25       // Tránh vòng lặp nếu đang ở trang login
26       if (!window.location.pathname.startsWith('/login')) {
27         window.location.href = '/login';
28       }
29     }
30     return Promise.reject(err);
31   }
32 );
33
34 export default api;

```

Listing 4.10: File AuthContext.jsx – Quản lý trạng thái đăng nhập và hàm hasRole

```

1 import { createContext, useContext, useState, useCallback } from 'react';
2 import api from './api';
3
4 const AuthContext = createContext(null);
5
6 export function AuthProvider({ children }) {
7   const [user, setUser] = useState(() => {
8     const raw = localStorage.getItem('user');
9     return raw ? JSON.parse(raw) : null;
10  });
11
12  const login = useCallback(async (TenDangNhap, MatKhai) => {
13    const res = await api.post('/api/auth/login', { TenDangNhap, MatKhai });
14    localStorage.setItem('token', res.data.token);
15    localStorage.setItem('user', JSON.stringify(res.data.user));
16    setUser(res.data.user);
17    return res.data.user;
18  }, []);
19
20  const logout = useCallback(() => {
21    localStorage.removeItem('token');
22    localStorage.removeItem('user');
23    setUser(null);
24  }, []);
25
26  // 'Quan ly' co toan quyen -- hasRole(...) kiem tra vai tro
27  const hasRole = useCallback(
28    (...roles) => !!user && (user.VaiTro === 'Quan ly' || roles.includes(user.VaiTro)),
29    [user]
30  );
31
32  return (
33    <AuthContext.Provider value={{ user, login, logout, hasRole }}>
34      {children}
35    </AuthContext.Provider>
36  );
37 }
38
39 export function useAuth() {
40   const ctx = useContext(AuthContext);
41   if (!ctx) throw new Error('useAuth phải dùng bên trong <AuthProvider>');
42   return ctx;
43 }

```

4.4 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu đặt tên là ShiftPOS, dùng MySQL với bộ ký tự utf8mb4 để lưu được tiếng Việt đầy đủ. Các bảng chính:

Bảng 4.3: Các bảng chính trong cơ sở dữ liệu

Bảng	Ý nghĩa	Trường tiêu biểu
KHUVUC	Khu vực	MaKV, TenKV
BAN	Bàn	MaBan, TenBan, TrangThai, MaKV
DANHMUC	Danh mục món	MaDM, TenDM
MONAN	Món ăn/thức uống	MaMon, TenMon, DonGia, TrangThai, MaDM
NHANVIEN	Nhân viên	MaNV, TenNV, VaiTro, MucLuong
TAIKHOAN	Tài khoản đăng nhập	TenDangNhap, MatKhou (hash), MaNV
HOADON	Hóa đơn	MaHD, TrangThaiHD, PhuongThucTT, ChietKhou, ThueVAT, ThoiGianMo, ThoiGianDong, MaBan, MaNV
CHITiet_HD	Chi tiết hóa đơn	MaHD, MaMon, SoLuong, DonGiaLucBan, TrangThaiMon, ThoiGianGoi
THUCHI	Phiếu thu/chi	MaPhieu, LoaiPhieu, SoTien, LyDo, NguoiLap

Listing 4.11: File KhoiTao.sql – Định nghĩa bảng HOADON và CHITiet_HD

```

1  -- KhoiTao.sql: Schema co so du lieu ShiftPOS
2  DROP DATABASE IF EXISTS ShiftPOS;
3  CREATE DATABASE ShiftPOS CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
4  USE ShiftPOS;
5
6  -- Bang hoa don: moi phien ban tai 1 ban
7  CREATE TABLE HOADON (
8      MaHD          INT AUTO_INCREMENT PRIMARY KEY,
9      ThoiGianMo    DATETIME DEFAULT CURRENT_TIMESTAMP,
10     ThoiGianDong   DATETIME NULL,
11     TrangThaiHD    VARCHAR(50) DEFAULT 'Chua thanh toan',
12     PhuongThucTT   VARCHAR(50) DEFAULT 'Tien mat',
13     ChietKhou      DECIMAL(5,2) DEFAULT 0,      -- % chiet khau
14     ThueVAT        DECIMAL(5,2) DEFAULT 0,      -- % VAT
15     MaBan          VARCHAR(10),
16     MaNV           VARCHAR(10),
17     FOREIGN KEY (MaBan) REFERENCES BAN(MaBan),
18     FOREIGN KEY (MaNV) REFERENCES NHANVIEN(MaNV)
19 );
20
21 -- Bang chi tiet: tung mon trong moi hoa don
22 CREATE TABLE CHITiet_HD (
23     MaHD           INT,
24     MaMon          VARCHAR(10),
25     SoLuong        INT,
26     DonGiaLucBan   INT,      -- gia tai thoi diem ban (bat bien khi doi gia sau)
27     GhiChu         VARCHAR(100),
28     TrangThaiMon   VARCHAR(30) DEFAULT 'Cho che bien', -- trang thai bep
29     ThoiGianGoi    DATETIME DEFAULT CURRENT_TIMESTAMP, -- dung de sap xep FIFO
30     PRIMARY KEY (MaHD, MaMon),
31     FOREIGN KEY (MaHD) REFERENCES HOADON(MaHD),

```

```

32 FOREIGN KEY (MaMon) REFERENCES MONAN (MaMon)
33 );

```

Các đối tượng CSDL bổ sung (trong ChucNang.sql):

- **Hàm fn_TinhTienBan:** tính tổng tiền tạm tính của một bàn bằng một câu SQL. Back-end gọi hàm này bất cứ khi nào cần hiển thị số tiền trên ô bàn, thay vì tính lại trong Python.
- **Thủ tục sp_ChuyenBan, sp_GopBan:** chứa logic chuyển/gộp hóa đơn giữa các bàn. Gom logic phức tạp vào CSDL để back-end chỉ cần gọi một lệnh.
- **Trigger trg_DonBan_SauThanhToan:** tự động chạy ngay khi hóa đơn chuyển sang “Đã thanh toán”. Trigger này ghi thời gian đóng và kiểm tra: nếu bàn đó không còn hóa đơn nào đang mở thì tự trả bàn về “Trống”. Nhờ trigger, back-end không cần viết thêm bước này sau mỗi lần thanh toán.
- **View vw_DoanhThuTheoNgay, vw_BaoCaoThuChi:** câu truy vấn tổng hợp sẵn để phân báo cáo dùng, tránh lặp SQL phức tạp ở nhiều chỗ.

Listing 4.12: File ChucNang.sql – Hàm fn_TinhTienBan và Trigger trg_DonBan_SauThanhToan

```

1  -- [FO1] fn_TinhTienBan: Tính tiền tạm tính của một bàn
2  DELIMITER //
3  CREATE FUNCTION fn_TinhTienBan(p_MaBan VARCHAR(10))
4  RETURNS INT DETERMINISTIC
5  BEGIN
6      DECLARE Tong INT;
7      SELECT SUM(CT.SoLuong * CT.DonGiaLucBan) INTO Tong
8      FROM HOADON H JOIN CHITIET_HD CT ON H.MaHD = CT.MaHD
9      WHERE H.MaBan = p_MaBan AND H.TrangThaiHD = 'Chưa thanh toán';
10     RETURN IFNULL(Tong, 0);
11 END //
12
13 -- [TO1] trg_DonBan_SauThanhToan (BEFORE UPDATE ON HOADON)
14 -- Khi hóa đơn -> 'Đã thanh toán': set ThoiGianDong, trả bàn về 'Trống'
15 -- nếu không còn hóa đơn nào đang mở trên bàn đó.
16 CREATE TRIGGER trg_DonBan_SauThanhToan
17 BEFORE UPDATE ON HOADON
18 FOR EACH ROW
19 BEGIN
20     IF NEW.TrangThaiHD = 'Đã thanh toán'
21     AND OLD.TrangThaiHD <> 'Đã thanh toán' THEN
22         SET NEW.ThoiGianDong = CURRENT_TIMESTAMP;
23         -- Kiểm tra còn hóa đơn chưa thanh toán nào trên bàn không
24         IF (SELECT COUNT(*) FROM HOADON
25             WHERE MaBan = NEW.MaBan
26                 AND TrangThaiHD = 'Chưa thanh toán'
27                 AND MaHD <> NEW.MaHD) = 0 THEN
28             UPDATE BAN SET TrangThai = 'Trống' WHERE MaBan = NEW.MaBan;
29         END IF;

```

```

30     END IF;
31 END //
32 DELIMITER ;

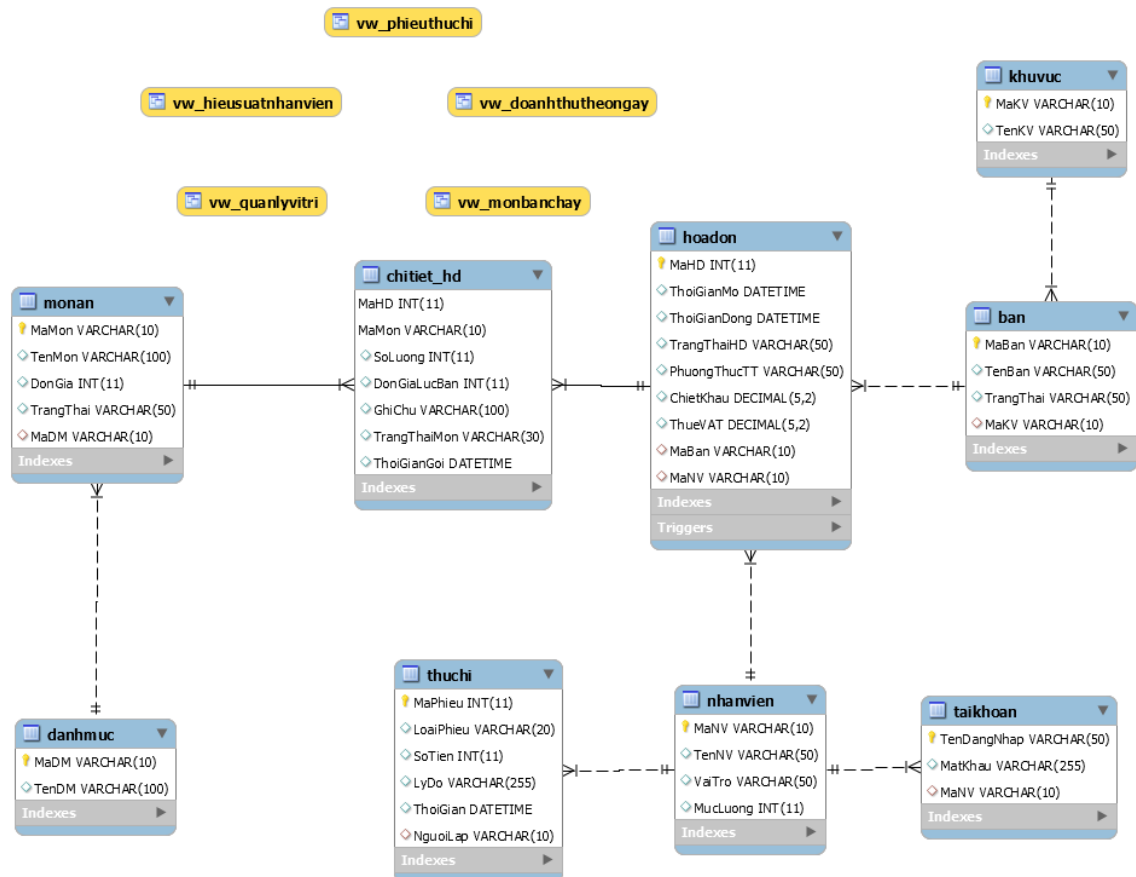
```

Các giá trị trạng thái lưu trong CSDL:

- Bàn: 'Trống' (chưa có khách) hoặc 'Đang phục vụ' (đang có hóa đơn mở).
- Hóa đơn: 'Chưa thanh toán' hoặc 'Đã thanh toán'.
- Món trong hóa đơn: 'Chờ chế biến' → 'Đang chế biến' → 'Xong'.
- Hình thức thanh toán: 'Tiền mặt' hoặc 'Chuyển khoản'.

Một lưu ý về thiết kế: Cột `DonGiaLucBan` lưu giá tại thời điểm khách gọi món, không tham chiếu lại bảng `MONAN`. Điều này đảm bảo: nếu sau này chủ quán đổi giá món, những hóa đơn cũ vẫn giữ đúng số tiền ban đầu – giống như hóa đơn giấy đã in ra không thể sửa ngược lại.

Sơ đồ EER mô tả đầy đủ các thực thể, thuộc tính và mối quan hệ giữa các bảng trong cơ sở dữ liệu ShiftPOS:

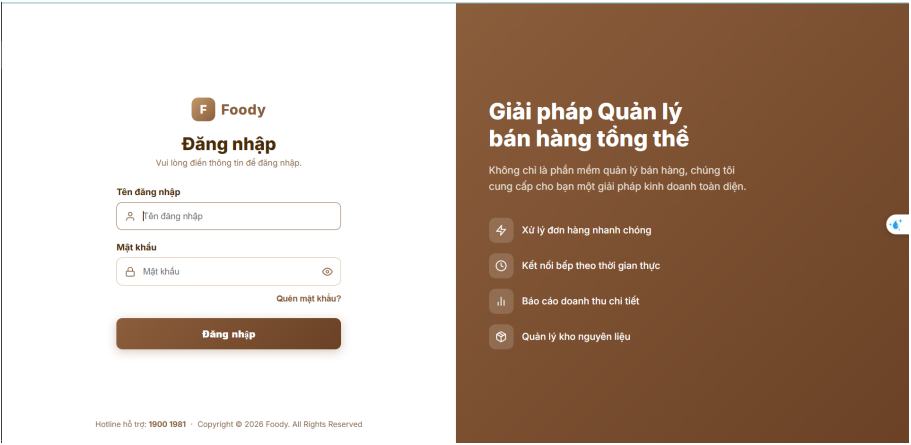


Hình 4.1: Sơ đồ EER – Cơ sở dữ liệu ShiftPOS

4.5 Thiết kế giao diện hệ thống

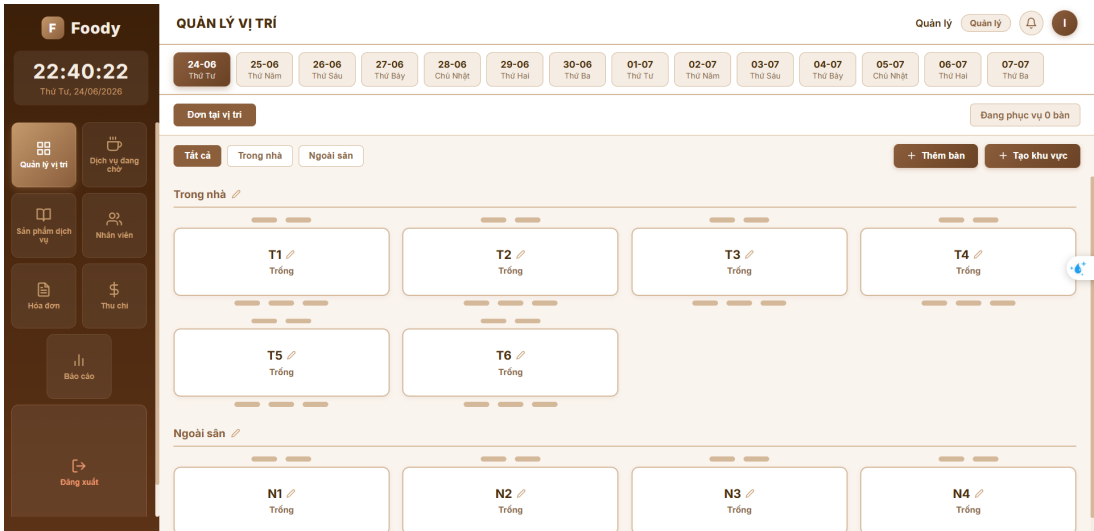
Giao diện dùng tông màu nâu – kem ấm áp, gợi cảm giác của quán cà phê hay nhà hàng (màu chủ đạo #8B5E3C, nền #FAF4EE, bàn đang có khách hiện cam #E8956D). Dưới đây là các màn hình chính theo từng vai trò.

Màn hình đăng nhập: nhập tên tài khoản và mật khẩu. Nếu thông tin đúng, hệ thống tự chuyển vào ứng dụng và nhớ đăng nhập cho đến khi hết 12 giờ hoặc bấm đăng xuất.



Hình 4.2: Màn hình đăng nhập của hệ thống

Màn hình POS – sơ đồ bàn: bên trái là toàn bộ bàn trong quán, chia theo khu vực, mỗi ô bàn hiện tên và số tiền đang tạm tính. Bàn trống màu nhạt, bàn đang có khách hiện cam sậm. Bên phải là thực đơn và hóa đơn của bàn đang chọn.



Hình 4.3: Màn hình POS chính — sơ đồ bàn theo khu vực

Popup thanh toán: thu ngân chọn hình thức thanh toán, nhập chiết khấu và VAT

nếu có, hệ thống tự tính tiền thực thu. Sau khi xác nhận có thể in hóa đơn ra máy in.

Thanh toán hóa đơn

Chuyển khoản

Tiền mặt

Chiết khấu (%)

VAT (%)

0

0

FOODY POS

Mã HĐ: 7

Vị trí: T1 · 22:42 24/06/2026

TPBank

Bún Cá Niêu Gạo Lứt x1

55.000

Bún Nghệ Cá Sả Sùng x1

55.000

Tạm tính:

110.000 đ

Chiết khấu (0%):

-0 đ

VAT (0%):

+0 đ

Khách cần trả:

110.000 đ

Thanh toán hóa đơn

Chuyển khoản

Tiền mặt

Chiết khấu (%)

VAT (%)

0

0

FOODY POS

Mã HĐ: 7

Vị trí: T1 · 22:46 24/06/2026

Bún Cá Niêu Gạo Lứt x1

55.000

Bún Nghệ Cá Sả Sùng x1

55.000

Tạm tính:

110.000 đ

Chiết khấu (0%):

-0 đ

VAT (0%):

+0 đ

Khách cần trả:

110.000 đ

In biên lai

Xác nhận thanh toán

In biên lai

Xác nhận thanh toán

Hình 4.4: Modal thanh toán hóa đơn

Màn hình Bếp: hiện toàn bộ món của các bàn đang phục vụ, sắp xếp theo thứ tự thời gian gọi – bàn nào gọi trước hiện trước. Nhân viên bếp bấm chuyển trạng thái từng món khi bắt đầu nấu và khi xong.

DỊCH VỤ ĐANG CHỜ					Quản lý	Quản lý		1
Dịch vụ đang chờ					12 món đang chờ biến			
Tên món	Loại	Bàn	Thời gian tạo	Trạng thái hoàn thành		Thanh toán		
Bún Cá Niêu Gạo Lứt × 1	Đồ ăn	T1	05:42 25-06	Đang chờ biến	Hoàn thành	Chưa thanh toán		
Bún Nghệ Cá Sả Sùng × 1	Đồ ăn	T1	05:42 25-06	Đang chờ biến	Hoàn thành	Chưa thanh toán		
Bún Nghệ Cá Sả Sùng Đặc Biệt × 1	Đồ ăn	T1	05:52 25-06	Đang chờ biến	Hoàn thành	Chưa thanh toán		
Bún Cá Niêu Gạo Lứt Đặc Biệt × 1	Đồ ăn	T1	05:52 25-06	Đang chờ biến	Hoàn thành	Chưa thanh toán		
Cacao Sữa × 2	Đồ uống	T1	05:52 25-06	Đang chờ biến	Hoàn thành	Chưa thanh toán		
Trà Sả Dứa × 1	Đồ uống	T3	05:52 25-06	Đang chờ biến	Hoàn thành	Chưa thanh toán		
Miến Khoai Lang Cá Tầm × 1	Đồ ăn	T3	05:52 25-06	Đang chờ biến	Hoàn thành	Chưa thanh toán		
Cafe Đen Đá × 1	Đồ uống	N3	05:52 25-06	Đang chờ biến	Hoàn thành	Chưa thanh toán		
Chà Mọc Cá × 1	Món gọi thêm	N3	05:52 25-06	Đang chờ biến	Hoàn thành	Chưa thanh toán		

Hình 4.5: Màn hình Bếp (Kitchen Display) — hàng đợi món theo thứ tự FIFO

Màn hình Hóa đơn: lịch sử toàn bộ hóa đơn đã thanh toán, có thể lọc theo ngày và

phương thức thanh toán.

HÓA ĐƠN Quản lý Quản lý 🔔 1

Tất cả vị trí ▼ Tất cả nhân viên ▼ Tất cả thanh toán ▼ dd/mm/yyyy → dd/mm/yyyy

Tổng tiền (8 hóa đơn)
665.000 đ

#	Mã đơn hàng	Nhân viên	Số SP/DV	Thời gian tạo	Thời gian thanh toán	Tổng tiền	Vị trí	PT thanh toán	Trạng thái
1	#9	Quản lý	2	22:52 24-06	22:56 24-06	70.000 đ	T3	\$ Tiền mặt	Đã thanh toán
2	#7	Quản lý	2	22:42 24-06	22:56 24-06	110.000 đ	T1	🏠 Chuyển khoản	Đã thanh toán
3	#6	Quản lý	1	15:41 18-06	16:54 24-06	5.000 đ	T1	🏠 Chuyển khoản	Đã thanh toán
4	#5	Quản lý	2	15:39 18-06	15:39 18-06	110.000 đ	T1	\$ Tiền mặt	Đã thanh toán
5	#4	Quản lý	2	15:39 18-06	15:39 18-06	140.000 đ	T1	🏠 Chuyển khoản	Đã thanh toán
6	#3	Quản lý	1	15:06 18-06	15:06 18-06	65.000 đ	T1	🏠 Chuyển khoản	Đã thanh toán
7	#2	Quản lý	1	15:06 18-06	15:06 18-06	85.000 đ	T1	\$ Tiền mặt	Đã thanh toán
8	#1	Quản lý	2	15:04 18-06	15:04 18-06	80.000 đ	T1	🏠 Chuyển khoản	Đã thanh toán

Hình 4.6: Màn hình Hóa đơn — lịch sử thanh toán

Màn hình Thực đơn: hiển thị các món chia theo danh mục, Quản lý thêm/sửa/xóa món và danh mục trực tiếp tại đây.

SẢN PHẨM DỊCH VỤ Quản lý Quản lý 🔔 1

🔍 Tìm SP Dịch vụ

Tất cả Đồ ăn Đồ uống Món gọi thêm + Danh mục + Thêm món

 Đồ uống Cafe Đen Đá Giá: 25.000đ	 Món gọi thêm Chả Mọc Cá Giá: 25.000đ	 Đồ ăn Bún Nghệ Cá Sả S... Giá: 85.000đ	 Đồ uống Bạc Xiu Giá: 30.000đ	 Đồ uống Trà Đào Cam Sả Giá: 35.000đ
 Đồ uống Cacao Sữa Giá: 30.000đ	 Đồ uống Trà Sấm Dứa Giá: 5.000đ	 Đồ ăn Bún Cá Niêu Gạo ... Giá: 55.000đ	 Đồ ăn Bún Nghệ Cá Sả S... Giá: 55.000đ	 Đồ ăn Miến Khoai Lang ... Giá: 65.000đ
 Đồ ăn Bún Cá Niêu Gạo ... Giá: 85.000đ	 Món gọi thêm Cá Tầm Thêm Giá: 35.000đ			

Hình 4.7: Màn hình quản lý thực đơn và danh mục

Màn hình Nhân viên: Quản lý xem danh sách nhân viên kèm tên tài khoản đăng nhập, thêm người mới, sửa thông tin, hoặc đặt lại mật khẩu khi nhân viên quên.

QUẢN LÝ NHÂN VIÊN						Quản lý	Quản lý		
Quản lý Nhân viên						+ Thêm Nhân viên			
Tạo và quản lý tài khoản nhân viên									
Mã NV	Họ và tên	Vai trò	Tên đăng nhập	Lương cơ bản	Thao tác				
BEP01	Thu	Bếp	thu	10.000.000 đ	Sửa	Đổi MK	Xóa		
NV01	Quản lý	Quản lý	admin	15.000.000 đ	Sửa	Đổi MK	Xóa		
NV02	Ngân	Thu ngân	ngan	7.500.000 đ	Sửa	Đổi MK	Xóa		
NV03	Dũng	Phục vụ	dung	7.500.000 đ	Sửa	Đổi MK	Xóa		

Hình 4.8: Màn hình quản lý nhân viên và tài khoản

Màn hình Thu Chi: ghi các khoản thu/chi ngoài hóa đơn – ví dụ tiền nhập hàng, tiền điện, hay khoản thu lẻ. Cuối trang tự tổng kết tổng thu và tổng chi.

SỔ THU CHI

Quản lý

Quản lý

1

Sổ Thu / Chi

Ghi các khoản tiền phát sinh ngoài bán hàng — Thu: tiền tip, khách trả thiếu... - Chi: chi phí vặt. Các khoản này không tính vào doanh thu ở báo cáo.

Tất cả loại phiếu

Tất cả người tạo

dd/mm/yyyy

→

dd/mm/yyyy

Tổng thu

600.000 đ

Tổng chi

225.000 đ

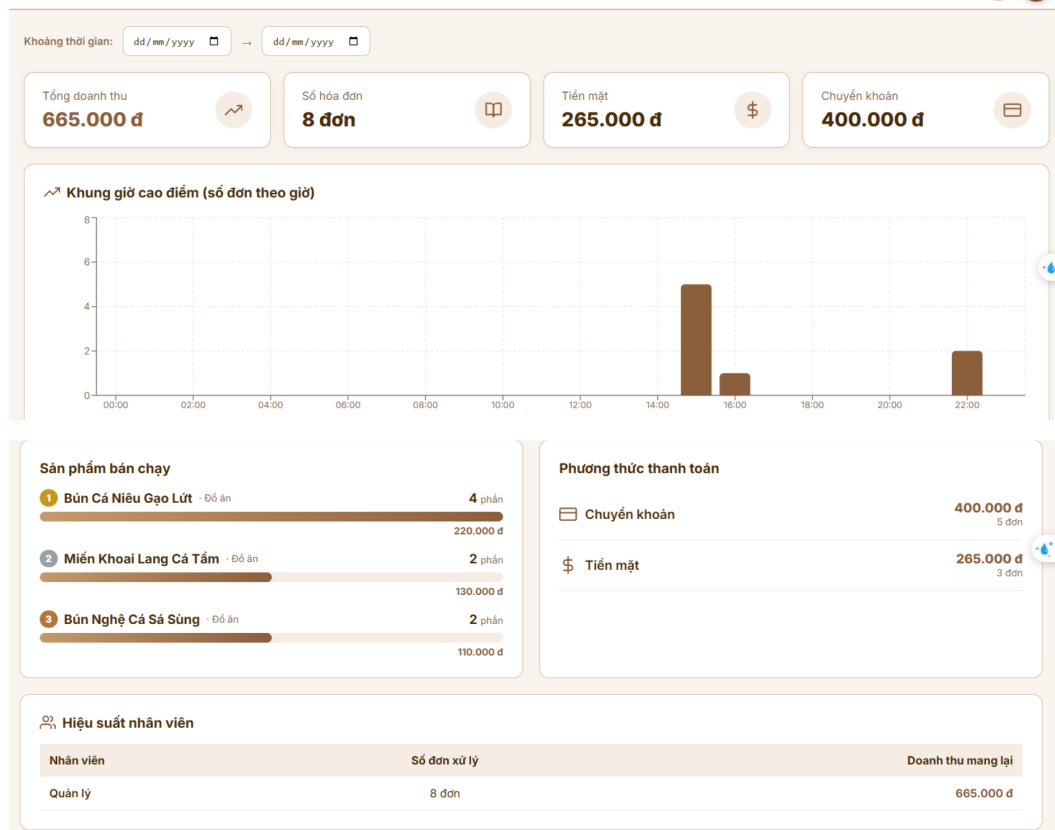
Danh sách phiếu

+ Tạo phiếu thu/chi

Loại phiếu	Mã phiếu	Người tạo phiếu	Khoản mục	Ngày tạo	Số tiền	Thao tác
Phiếu thu	#4	Quản lý	Nhận thêm tiền lẻ của chủ quán	06:42 25/06/2026	+500.000 đ	
Phiếu chi	#3	Quản lý	Sửa chữa camera	06:42 25/06/2026	-200.000 đ	
Phiếu thu	#2	Quản lý	Khách tip 100k phục vụ tốt	06:41 25/06/2026	+100.000 đ	
Phiếu chi	#1	Quản lý	Mua rau - mua rau	22:40 18/06/2026	-25.000 đ	

Hình 4.9: Màn hình sổ Thu – Chi

Màn hình Báo cáo: Quản lý lọc theo khoảng ngày để xem: tổng doanh thu, biểu đồ doanh thu theo từng giờ trong ngày (để biết giờ nào đông nhất), top 3 món bán chạy nhất, và bảng hiệu suất từng nhân viên.



Hình 4.10: Màn hình Báo cáo – thống kê doanh thu và hiệu suất

Chương 5

HIỆU QUẢ SỬ DỤNG VÀ HƯỚNG PHÁT TRIỂN

5.1 Hiệu quả sử dụng

So sánh trực tiếp với cách quản lý bằng giấy tờ:

- **Không còn nhầm order:** Món gọi từ bàn nào, do nhân viên nào tạo đều được lưu rõ. Bếp nhìn màn hình thay vì giải mã chữ viết tay.
- **Bếp và phục vụ không cần hét nhau:** Món vừa gọi hiện ngay trên màn hình bếp theo đúng thứ tự thời gian. Bếp cập nhật “Xong”, phục vụ biết đi lấy món mang ra.
- **Thanh toán nhanh hơn:** Không cần ngồi cộng tay. Thu ngân mở hóa đơn bàn, bấm thanh toán, bàn tự trống – cả quy trình dưới 30 giây.
- **Chủ quán biết con số thực:** Cuối ngày vào Báo cáo là thấy ngay doanh thu, giờ nào bận nhất, món nào đang chạy, nhân viên nào năng suất – không cần ngồi tổng hợp từ đồng hóa đơn giấy.
- **Không tốn thêm thiết bị:** Chạy trên trình duyệt, máy tính hay máy tính bảng đang có trong quán dùng được ngay.

5.2 Hướng phát triển trong tương lai

Những thứ mà phiên bản này chưa làm được nhưng hoàn toàn có thể bổ sung sau:

- **Thanh toán QR thực tế:** kết nối với ngân hàng/ví điện tử để sinh mã QR theo từng hóa đơn thay vì dùng ảnh tĩnh như hiện tại.
- **Quản lý kho:** theo dõi nguyên liệu, tự trừ kho khi bán món, cảnh báo khi gần hết hàng.

- **Ứng dụng điện thoại cho phục vụ:** dùng điện thoại gọi món trực tiếp tại bàn thay vì chạy về quầy.
- **Khách tự gọi món:** khách quét QR tại bàn và đặt món từ điện thoại của họ.
- **Nhiều chi nhánh:** hiện tại chạy được cho một địa điểm; mở rộng để quản lý chuỗi quán với báo cáo tổng hợp.
- **Test tự động:** thêm bộ kiểm thử code tự chạy để phát hiện lỗi sớm khi thêm tính năng mới.

Chương 6

KẾT LUẬN

6.1 Tóm tắt quá trình hoàn thành

Em bắt đầu từ việc quan sát cách một số quán ăn nhỏ đang làm việc và xác định những điểm bất tiện cần giải quyết. Từ đó, em thiết kế cơ sở dữ liệu, xây dựng API phía máy chủ, rồi dựng giao diện – theo thứ tự: màn hình POS cơ bản → đăng nhập và phân quyền → màn hình bếp → các thao tác nâng cao (chuyển/gộp/tách bàn) → báo cáo.

Mỗi tính năng sau khi xây xong được kiểm thử thủ công theo đúng quy trình thực tế: phục vụ gọi món → bếp cập nhật trạng thái → thu ngân thanh toán → quản lý kiểm tra báo cáo.

6.2 Kết quả đạt được

Phần mềm Foody đã hoạt động đầy đủ với toàn bộ tính năng đã đề ra:

- Đăng nhập và phân quyền 4 vai trò – kiểm soát ở cả máy chủ lẫn giao diện.
- Màn hình POS: sơ đồ bàn, gọi món, tạm tính, chuyển/gộp bàn, tách hóa đơn.
- Thanh toán hỗ trợ chiết khấu và VAT; bàn tự trống sau khi thanh toán.
- Màn hình bếp hiển thị hàng đợi FIFO và cập nhật trạng thái chế biến.
- Quản trị thực đơn, nhân viên và sổ thu/chi.
- Báo cáo doanh thu, giờ cao điểm, top món bán chạy, hiệu suất nhân viên.
- Giao diện tiếng Việt, in được hóa đơn ra máy in.

6.3 Đánh giá kết quả

Điểm làm được tốt:

- Bám sát quy trình thực tế của một quán ăn – không chỉ gọi món và thanh toán mà còn xử lý được các tình huống hay gặp như đổi bàn, nhóm nhập chung, tách tiền riêng.
- Code tổ chức rõ ràng – mỗi chức năng một file, dễ tìm và dễ sửa sau này.
- Dùng trigger trong cơ sở dữ liệu để xử lý logic bàn trống – đảm bảo trạng thái dữ liệu luôn đúng dù có nhiều người cùng thao tác.

Còn thiếu:

- Chưa có quản lý kho nguyên liệu.
- Thanh toán QR hiện chỉ là hình minh họa, chưa kết nối thực tế.
- Kiểm thử vẫn làm tay, chưa có code test tự động.

Nhìn lại, em thấy dự án đạt được những gì đề ra ban đầu. Quan trọng hơn, em hiểu sâu hơn về cách xây dựng một ứng dụng web thực sự – không chỉ viết code chạy được, mà còn phải nghĩ đến luồng nghiệp vụ, phân quyền, tính nhất quán dữ liệu và trải nghiệm người dùng.

— Hết —

TÀI LIỆU THAM KHẢO

- [1] A. K. Sharma, R. Kumar, and S. Singh, “Design and Implementation of a Cloud-Based Point-of-Sale System for Small Restaurant Businesses,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 13, no. 4, pp. 178–185, 2022.
- [2] H. T. Nguyen, M. Q. Tran, and L. V. Pham, “Applying RESTful API Architecture in Building a Web-Based Restaurant Ordering and Management System,” in *Proc. IEEE RIVF International Conference on Computing and Communication Technologies*, Hanoi, Vietnam, 2023, pp. 1–6.
- [3] C. Borza and M. Frentu, “Digital Transformation in Food Service Industry: A Study on POS System Adoption and Workflow Efficiency,” *Procedia Computer Science*, vol. 221, pp. 893–900, 2023.
- [4] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, Dept. of Information and Computer Science, University of California, Irvine, CA, USA, 2000.
- [5] L. Richardson and S. Ruby, *RESTful Web Services*. Sebastopol, CA: O’Reilly Media, 2007.
- [6] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-Based Access Control Models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [7] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force (IETF), May 2015. [Online]. Available: <https://datatracker.ietf.org/doc/rfc7519/>
- [8] D. Flanagan, *JavaScript: The Definitive Guide*, 7th ed. Sebastopol, CA: O’Reilly Media, 2020.
- [9] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ: Pearson Education, 2015.
- [10] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall, 2008.